

DD: FUL-02-STEP-CANCELLATION — Order Cancellation Cascade

Developer implementation guide: DD_FUL-02-STEP-CANCELLATION-DEV_IMPL_GUIDE-v1.0.md.

1. Purpose

This rewritten DD is the developer-facing version of the module contract DD_FUL-02-STEP-CANCELLATION-v1.0.md.

Verdict: ■ New | **Wave:** 1 | **Group III:** Fulfillment | **Version:** v1.0 | **Satellite of:** DD_FUL-02-Order_Capture-v1.0.md Handles cancellation of an in-flight order, from any phase prior to ACTIVATED. Triggered by customer-initiated request, ops decision, or payment-timeout. Owns the cascade logic — cleaning up downstream artifacts (subscription, install work order, payments) that may have been created during the journey, in reverse order, and emitting events that finance and the customer can act on. Phase: CANCELLED.

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/06_fulfillment/DD_FUL-02-STEP-CANCELLATION-v1.0.md
Version	v1.0
DD type	module contract
BPMN/process keys	Not explicitly defined
Drools packages	Not explicitly defined

4. Runtime Contracts To Implement

4.1 APIs

- POST /api/admin/work-orders/wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2/cancel
- POST /api/admin/work-orders/{id}/cancel
- POST /api/orders/ord_01HZ.../cancel
- POST /api/orders/ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1/cancel
- POST /api/orders/{id}/cancel
- POST /api/orders/{orderId}/cancel
- POST /api/subscriptions/sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q/terminate

- POST /api/subscriptions/{subscription_id}/terminate
- POST /engine-rest/message

4.2 Tables

- No CREATE TABLE block is explicitly declared in this DD.

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- cancel-requested
- cascade-cancellation
- mark-cancelled
- order-cascade-cancellation
- order-mark-cancelled
- wait-install-finalize
- wait-payment

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- WorkOrderCancelled
- WorkOrderCancelledConsumer

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.

10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 1 | **Group III:** Fulfillment | **Version:** v1.0 | **Satellite of:** DD_FUL-02-Order_Capture-v1.0.md

Handles cancellation of an in-flight order, from any phase prior to ACTIVATED. Triggered by customer-initiated request, ops decision, or payment-timeout. Owns the cascade logic — cleaning up downstream artifacts (subscription, install work order, payments) that may have been created during the journey, in reverse order, and emitting events that finance and the customer can act on. Phase: CANCELLED.

1. What this step does

A customer's order can be cancelled at any time before the order reaches ACTIVATED. The trigger can come from:

- **Customer-initiated cancellation** — the customer calls support and asks to cancel, and a support agent issues the cancel from the BO UI.
- **Payment timeout** — the PT7D boundary timer on wait-payment fires (STEP-PAYMENT).
- **Ops escalation** — an ops user manually cancels an order via the admin UI (e.g., to resolve a fraud case).

The cancellation entry point varies (REST API call vs Camunda timer fire), but they all converge on the same order-cascade-cancellation worker. That worker walks the order's current state to figure out what's been created so far, cleans each up in reverse creation order, then order-mark-cancelled finalizes the order with phase CANCELLED.

The reverse-creation order matters because each cleanup depends on the previous one not being in a half-state:

1. **Cancel the install WO** (if `work_order_id` IS NOT NULL and the order is in INSTALLING or beyond). Calls the WO module to set the WO to CANCELLED — contractor is no longer dispatched, equipment is no longer needed. v1.0 routes to the `install_wo_stub` admin cancel endpoint.
2. **Terminate the subscription** (if `subscription_id` IS NOT NULL). Calls SUB-WF-TERMINATE-01 to walk the subscription from CREATED (or PENDING_ACTIVATION, in race scenarios) to TERMINATED. SUB-WF-TERMINATE-01 owns its own saga (calls BIL-01 to close any open invoices, NMS to deactivate any provisioning, etc.).
3. **Request refund** (if `payment_received_amount` > 0). Emits OrderRefundRequested for BIL-02-ADJ-01 to process. Same refund cascade as STEP-INSTALL / STEP-ACTIVATION failures. FUL-02 doesn't issue the refund itself — Finance does, on the refund queue.

Each cleanup is conditional: if a step never ran, its artifacts don't exist, no cleanup needed. The worker checks NULL on each column on `customer_order` before invoking the relevant cleanup.

How the customer-initiated cancel API works

A support agent (via BO UI) calls POST /api/orders/{orderId}/cancel:

```
POST /api/orders/ord_01HZ.../cancel HTTP/1.1
Authorization: Bearer <support JWT, role ORDER_SUPPORT>
Content-Type: application/json

{
  "reason":      "CUSTOMER_REQUESTED",
  "detail":      "Customer changed mind – moving to a different ISP",
  "actor":       "cust_01HX..."
}
```

OrderCaptureService:

1. Verifies the order exists and is not already terminal (REJECTED/CANCELLED/ACTIVATED/INSTALL_FAILED/ACTIVATION_FAILED → 409 with ORDER_ALREADY_TERMINAL).
2. Verifies the order isn't ACTIVATED — if it is, the customer should use the terminate-subscription flow instead (different code path, different refund rules, etc.). Returns 409 ORDER_ALREADY_ACTIVATED_USE_TERMINATE pointing at SUB-WF-TERMINATE-01.
3. Sends a Camunda message correlation directly (no Kafka — this is a synchronous control plane action):

```
POST /engine-rest/message HTTP/1.1
{
  "messageName": "OrderCancellationRequested",
  "businessKey": "ord_01HZ...",
  "processVariables": {
    "cancellationReason": { "value": "CUSTOMER_REQUESTED", "type": "String" },
    "cancellationDetail": { "value": "Customer changed mind – moving to a different ISP", "type": "String" },
    "cancellationActor": { "value": "cust_01HX...", "type": "String" }
  }
}
```

The BPMN's top-level interrupting boundary message event (FRAMEWORK §1.8) catches OrderCancellationRequested and interrupts whatever subprocess the order was in — be it wait-payment, the L1 KYC user task, the Team Leader KYC user task, or the install wait. Control routes to order-cascade-cancellation.

For audit, the cancellation request itself is recorded in a side table customer_order_cancellation_requests BEFORE the Camunda call (so it survives even if Camunda 5xxs).

Entry path: Camunda message, not Kafka

The cancel entry uses Camunda's REST message API directly, not Kafka. Reason: cancellation is a control-plane action that needs to be confirmed synchronously to the caller (support agent expects an immediate "yes, your order was cancelled" or "couldn't cancel because X"). Kafka would add eventual-consistency delay and remove the ability to return a meaningful error code. Note that the *outcome* events (OrderCancelled, OrderRefundRequested) ARE emitted on Kafka — those are downstream notifications, not control-plane handshakes.

What the customer journey looks like during cancellation

When order-cascade-cancellation and order-mark-cancelled complete, the customer receives a DD_NOT-01 notification:

- If money was received and refund was requested: "Your order has been cancelled. You'll receive a refund of {amount} within 5 business days."
- If no money was received: "Your order has been cancelled. No refund needed."
- If the cancellation was system-initiated (payment timeout): "Your order was cancelled because we did not receive payment within 7 days. {refund text if applicable}."

What this step deliberately doesn't do

- **Doesn't cancel an ACTIVATED order.** Once the order has reached ACTIVATED, the subscription is live; cancelling it is a subscription-termination operation (SUB-WF-TERMINATE-01), not a FUL-02 concern. The cancel API returns 409.
- **Doesn't issue the actual refund.** That's BIL-02-ADJ-01 (consumes OrderRefundRequested events into a Finance queue). FUL-02's job is to declare that a refund is owed.
- **Doesn't roll back the customer record.** The customer_stub row (v1.0) or ILM-CFG-01 record (future) survives — the customer can re-submit a new order with the same identity.

2. Worked example — customer cancel during Awaiting_Payment

Scenario: John captured his order at 09:00 (WIK). STEP-VALIDATE passed at 09:00:01; STEP-SUBSCRIPTION created sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q at 09:00:02; STEP-PAYMENT computed 15000 KES and emitted instructions at 09:00:03. Camunda parked at wait-payment. At 10:15 John calls support and asks to cancel — changed mind.

Current order state:

- subscription_id = 'sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q' (CREATED status; subscription exists)
- work_order_id = NULL (no install WO yet)
- payment_received_amount = 0.00 (no payment received yet)
- current_phase = Awaiting_Payment

Step 1 — Support agent calls the cancel API

Support agent in their BO UI clicks "Cancel order" on John's ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1. UI POSTs:

```
POST /api/orders/ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1/cancel HTTP/1.1
Authorization: Bearer <support JWT, role ORDER_SUPPORT>

{
  "reason": "CUSTOMER_REQUESTED",
  "detail": "Customer called support – changed mind, no longer interested",
  "actor": "support_agent_03"
}
```

Step 2 — OrderCaptureService validates and records

OrderCaptureService:

1. Read order: current_phase = Awaiting_Payment. Not terminal, not ACTIVATED ✓.
2. INSERT into audit:

```
INSERT INTO customer_order_cancellation_requests (
  id, order_id, requested_by_actor, requested_by_role, reason, detail, requested_at, status
) VALUES (
  'cocr_01HZ...', 'ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1', 'support_agent_03', 'ORDER_SUPPORT',
  'CUSTOMER_REQUESTED', 'Customer called support – changed mind, no longer interested',
  NOW(), 'REQUESTED'
);
```

3. Correlate Camunda message:

```
POST /engine-rest/message HTTP/1.1
{
  "messageName": "OrderCancellationRequested",
  "businessKey": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "processVariables": {
    "cancellationReason": { "value": "CUSTOMER_REQUESTED", "type": "String" },
    "cancellationDetail": { "value": "Customer called support – changed mind, no longer interested", "type": "String" },
    "cancellationActor": { "value": "support_agent_03", "type": "String" }
  }
}
```

4. Camunda 204. UPDATE audit: status = ACKNOWLEDGED.

5. Return 202 Accepted to the UI:

```
{
  "orderId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "status": "ACKNOWLEDGED",
  "message": "Cancellation requested; cascade in progress. Outcome will arrive on Kafka."
}
```

The 202 returns immediately; the actual cascade runs async via Camunda. The UI listens to `sophix.orders.cancelled` to know when the cancellation is fully complete.

Step 3 — Camunda boundary event interrupts wait-payment

The order's process instance is parked at wait-payment (waiting for John's M-Pesa). The interrupting boundary message event named cancel-requested on the top-level subprocess catches OrderCancellationRequested. Camunda interrupts the wait, routes to order-cascade-cancellation.

Step 4 — order-cascade-cancellation runs

CascadeCancellationWorker picks up the external task. Reads the order row to determine what needs cleanup:

```
SELECT subscription_id, work_order_id, payment_received_amount, expected_payment_currency
FROM customer_order
WHERE order_id = 'ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1';
```

→ subscription_id IS NOT NULL (subscription needs termination); other columns NULL or zero.

Worker walks the cleanup in reverse creation order:

a. **No install WO** (work_order_id IS NULL) → skip.

b. **Terminate the subscription** (subscription_id IS NOT NULL):

```
POST /api/subscriptions/sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q/terminate HTTP/1.1
Host: subscription-workflow-terminate.sophix.internal

{
  "correlationId": "term_ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "reason": "ORDER_CANCELLED",
  "reasonDetail": "FUL-02 order cancellation cascade – order CUSTOMER_REQUESTED",
  "channel": "FULFILLMENT_ORDER_CAPTURE",
  "actor": "fulfillment-order-capture-worker-11"
}
```

SUB-WF-TERMINATE-01 responds 202 Accepted with a processInstanceId. The terminate saga runs async (closes any open invoices, deactivates any provisioning, etc.). FUL-02 doesn't wait for it to fully complete — accepting the 202 means it's been initiated, and that's enough for FUL-02's order to transition to CANCELLED.

Rationale: waiting synchronously for the terminate saga would couple FUL-02's BPMN to SUB-WF-TERMINATE-01's runtime, slowing the customer-facing cancellation acknowledgement. The terminate's eventual completion is announced separately on sophix.subscription.subscription.terminated for anyone who needs it.

c. **No refund** (payment_received_amount = 0.00) → skip.

Worker reports completion:

```
{
  "workerId": "fulfillment-order-capture-worker-11",
  "variables": {
    "cascadeOutcome": {
      "value": "{\"subscriptionTerminated\":true,\"installWoCancelled\":false,\"refundRequested\":false}",
      "type": "Json"
    }
  }
}
```

Camunda advances to order-mark-cancelled.

Step 5 — order-mark-cancelled finalizes

MarkCancelledWorker:

```
UPDATE customer_order
SET current_phase = 'CANCELLED',
    failure_reason = 'CUSTOMER_REQUESTED',
    failure_detail = 'Customer called support – changed mind, no longer interested',
    last_updated_at = NOW()
WHERE order_id = 'ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1';
```

Updates cancellation audit row: status = COMPLETED.

Emits sophix.orders.cancelled:

```
{
  "eventType": "OrderCancelled",
  "aggregateId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "timestamp": "2026-05-18T10:15:32.123Z",
  "payload": {
    "orderId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "customerId": "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
    "operatorCode": "WIK",
    "cancellationReason": "CUSTOMER_REQUESTED",
    "cancellationDetail": "Customer called support – changed mind, no longer interested",
    "cancellationActor": "support_agent_03",
    "cancelledFromPhase": "AWAITING_PAYMENT",
    "cascadeOutcome": {
      "subscriptionTerminated": true,
      "installWoCancelled": false,
      "refundRequested": false
    }
  },
  "paymentReceivedAmount": 0.00,
}
```

```

    "cancelledAt": "2026-05-18T10:15:32Z"
  }
}

```

BPMN ends. DD_NOT-01 → John: "Your order has been cancelled. No payment was received; nothing further to do."

Total elapsed in step: ~1.2 seconds (subscription terminate accepted, message emitted, mark-cancelled written).

3. Worked example — customer cancel during INSTALLING (full cascade with refund)

Variant: same customer, but cancels at 14:00 — after KYC cleared at 11:42 and the install WO was created at 12:00. The contractor hasn't arrived yet.

Order state at the moment of cancel

- subscription_id = 'sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q' (CREATED status).
- work_order_id = 'wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2' (install WO, status CREATED).
- payment_received_amount = 15000.00 (paid at 11:23).
- current_phase = INSTALLING.

Step 4 (variant) — order-cascade-cancellation runs

Worker reads the order, sees all those fields set. Walks the cleanup in reverse creation order:

a. **Cancel the install WO** (work_order_id = 'wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2' IS NOT NULL):

```

POST /api/admin/work-orders/wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2/cancel HTTP/1.1
{
  "cancellationReason": "ORDER_CANCELLED",
  "cancellationDetail": "FUL-02 order cancellation cascade — order CUSTOMER_REQUESTED",
  "cancelledBy": "fulfillment-order-capture-worker-11"
}

```

WO stub UPDATES install_wo_stub row: status=CANCELLED. Emits WorkOrderCancelled on Kafka. But: WorkOrderCancelledConsumer normally feeds back into the BPMN's wait-install-finalize boundary to fail the order — since the BPMN's main subprocess is already interrupted (by OrderCancellationRequested), the message correlation won't find an active subprocess. Camunda returns 400. Consumer DLQs the event (R-FUL-02-CAN-7) — the cascade in progress already handles the cancel; no double-handling.

b. **Terminate the subscription** (subscription_id IS NOT NULL):

```

POST /api/subscriptions/sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q/terminate HTTP/1.1
{ "correlationId": "term_ord_01HZ...", "reason": "ORDER_CANCELLED", ... }

```

202 Accepted. Async.

c. **Request refund** (payment_received_amount = 15000.00 > 0):

Emit sophix.orders.refund-requested:

```

{
  "eventType": "OrderRefundRequested",
  "aggregateId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "timestamp": "2026-05-18T14:00:33.412Z",
  "payload": {
    "orderId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "customerId": "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
    "operatorCode": "WIK",
    "subscriptionId": "sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q",
    "paymentReceivedAmount": 15000.00,
    "currency": "KES",
    "cancellationReason": "CUSTOMER_REQUESTED",
    "originatingStep": "STEP-CANCELLATION",
    "requestedAt": "2026-05-18T14:00:33Z"
  }
}

```

BIL-02-ADJ-01 consumes and queues the refund for Finance.

Worker reports cascadeOutcome:

```

{ "subscriptionTerminated": true, "installWoCancelled": true, "refundRequested": true }

```


Step 5 (variant) — order-mark-cancelled

Same as §2.5 but `cancelledFromPhase = INSTALLING`, `paymentReceivedAmount = 15000.00`, and `cascadeOutcome.refundRequested = true`. DD_NOT-01 → John: "Your order has been cancelled. You'll receive a refund of 15000 KES within 5 business days."

4. Worked example — auto-cancellation from PT7D payment timer

The boundary timer on `wait-payment` fires at 2026-05-25T09:00:03Z. Camunda routes the flow to `order-cascade-cancellation` directly (no API call, no audit row in `customer_order_cancellation_requests` — auto-triggered).

`order-cascade-cancellation` runs with `cancellationReason = PAYMENT_TIMEOUT` (set by the BPMN's gateway after the timer fires). Cleanup: subscription was created at STEP-SUBSCRIPTION, so it gets terminated.

`payment_received_amount = 0` → no refund. `work_order_id` is NULL → no WO cleanup.

`order-mark-cancelled` runs with `cancellationReason = PAYMENT_TIMEOUT`, `cancelledFromPhase = AWAITING_PAYMENT`. The `OrderCancelled` event's `cancellationActor` is set to `system_timer` to distinguish from human-initiated cancellations.

5. API endpoint

5.1 POST /api/orders/{orderId}/cancel

Customer-initiated or support-initiated cancellation.

Auth: role `CUSTOMER_SELF_SERVICE` (customer cancelling their own order — actor must be the order's `customer_id`) OR `ORDER_SUPPORT` (support agent acting on behalf of customer) OR `ORDER_ADMIN` (ops admin).

Request body:

Field	Required	Type	Plain-English
<code>reason</code>	yes	enum	Why the order is being cancelled. Enum: <code>CUSTOMER_REQUESTED</code> , <code>OPS_INITIATED</code> , <code>FRAUD_HOLD</code> , <code>OPERATOR_DECISION</code>
<code>detail</code>	yes	string	Free-text explanation. Audit + customer notification.
<code>actor</code>	yes	string	The user ID requesting cancellation. Must match JWT subject.

Response 202 Accepted:

```
{
  "orderId":      "ord_01HZ...",
  "cancellationId": "cocr_01HZ...",
  "status":       "ACKNOWLEDGED",
  "message":      "Cancellation requested; cascade in progress.",
  "eventTopicWatch": "sophix.orders.cancelled"
}
```

Error responses:

HTTP	Code	When
404	<code>ORDER_NOT_FOUND</code>	No order with this ID for the caller's operator scope
409	<code>ORDER_ALREADY_TERMINAL</code>	Phase is <code>REJECTED</code> / <code>CANCELLED</code> / <code>INSTALL_FAILED</code> / <code>ACTIVATION_FAILED</code>
409	<code>ORDER_ALREADY_ACTIVATED_USE_TERMINATE</code>	Phase is <code>ACTIVATED</code> — caller should use <code>SUB-WF-TERMINATE-01</code> 's subscription terminate endpoint instead
403	<code>ORDER_NOT_OWNED_BY_ACTOR</code>	<code>CUSTOMER_SELF_SERVICE</code> actor's <code>customerId</code> doesn't match the order's <code>customer_id</code>
503	<code>CAMUNDA_UNREACHABLE</code>	Camunda message correlation failed

6. Workers

6.1 order-cascade-cancellation

Field	Value
Topic	order-cascade-cancellation
Purpose	Walk the order's state in reverse creation order; clean up each downstream artifact that exists; produce a cascadeOutcome summary.
Process variables read	orderId, subscriptionId, workOrderId, cancellationReason, cancellationDetail, cancellationActor, paymentReceivedAmount, operatorCode
External HTTP calls	WO stub POST /api/admin/work-orders/{id}/cancel (per WO present). SUB-WF-TERMINATE-01 POST /api/subscriptions/{subscription_id}/terminate (if subscription present).
Process variables written	cascadeOutcome (JSON)
customer_order columns written	None directly — finalization is order-mark-cancelled's job.
Kafka emissions	sophix.orders.refund-requested (if payment received)
Lock duration	30 000 ms (multiple async calls inside)
Retry policy	3 retries with PT1M between attempts. Each downstream call is independently retried internally; the worker only /failures if a downstream call returns a permanent error.

Failure modes:

- **WO module unreachable** during install WO cancel → retry; if persistent, log the failure into `cascadeOutcome.installWoCancelled = false` and continue. The order still ends in CANCELLED phase; ops gets a NOC alert from the OrderCancelled event payload's cascadeOutcome flags.
- **SUB-WF-TERMINATE-01 4xx** (e.g., subscription already TERMINATED — race with another path) → treat as success; `subscriptionTerminated = true`.
- **SUB-WF-TERMINATE-01 5xx** → retry. After exhaustion: `cascadeOutcome.subscriptionTerminated = false`, log, continue. Ops gets alerted.

6.2 order-mark-cancelled

Field	Value
Topic	order-mark-cancelled
Purpose	Terminal handler for the cancel path. Marks the order CANCELLED, emits the canonical OrderCancelled event.
Process variables read	orderId, cancellationReason, cancellationDetail, cancellationActor, cascadeOutcome, operatorCode, cancelledFromPhase (set by BPMN gateway based on which step interrupted)
External HTTP calls	None
Process variables written	None (terminal)
customer_order columns written	current_phase = CANCELLED, failure_reason (= cancellationReason), failure_detail (= cancellationDetail)
customer_order_cancellation_requests columns written	status = COMPLETED (for the corresponding cocr_row, if any — auto-triggered cancellations have no cocr_row)
Kafka emissions	sophix.orders.cancelled
Lock duration	5 000 ms
Retry policy	3 retries with PT30S

7. DB tables touched

customer_order (FRAMEWORK §5.1) — UPDATE

Column	When set	Set by	Example value
current_phase	On BPMN entry to mark-cancelled	PhaseProjector	CANCELLED
current_activity_id	On Camunda transition	PhaseProjector	cascade-cancellation → mark-cancelled → end
failure_reason	order-mark-cancelled	CUSTOMER_REQUESTED / PAYMENT_TIMEOUT / OPS_INITIATED / FRAUD_HOLD / OPERATOR_DECISION	
failure_detail	order-mark-cancelled	Free-text from the request or the originating step	
last_updated_at	On every UPDATE	trigger / worker	2026-05-18 14:00:34.789

Sample data — five cancelled orders from various triggers:

order_id	operator_code	cancelled_from_phase	failure_reason	payment_received_amount	refund_requested
ord_01HZ...A	WIK	AWAITING_PAYMENT	CUSTOMER_REQUESTED	0.00	false
ord_01HZ...B	WIK	INSTALLING	CUSTOMER_REQUESTED	15000.00	true
ord_01HZ...C	WIK	AWAITING_PAYMENT	PAYMENT_TIMEOUT	0.00	false
ord_01HZ...D	WIK	IN_KYC	OPS_INITIATED (fraud investigation)	15000.00	true
ord_01HZ...E	WIK	INSTALLING	OPS_INITIATED (address change request — customer re-submits new order)	15000.00	true

customer_order_cancellation_requests — NEW AUDIT TABLE

Records every cancel API call before the Camunda correlation, so the audit survives even if Camunda crashes.

Column	Type	Notes
id	TEXT PK	ULID. Example: cocr_01HZX_CANCEL_01
order_id	TEXT NOT NULL	FK to customer_order. Example: ord_01HZ...
requested_by_actor	TEXT NOT NULL	User ID making the request. Example: support_agent_03
requested_by_role	TEXT NOT NULL	Values: ORDER_SUPPORT / ORDER_ADMIN. Example: ORDER_SUPPORT
reason	TEXT NOT NULL	Enum: CUSTOMER_REQUESTED / OPS_INITIATED / FRAUD_HOLD / OPERATOR_DECISION. Example: CUSTOMER_REQUESTED
detail	TEXT NOT NULL	Free-text. Example: Customer called support — changed mind, no longer interested
requested_at	TIMESTAMP NOT NULL	Example: 2026-05-18 10:15:32
status	TEXT NOT NULL	Lifecycle: REQUESTED → ACKNOWLEDGED (Camunda 204) → COMPLETED (cascade ran). Or REJECTED (validation failed: 4xx / 409).
status_detail	TEXT NULL	On REJECTED: the error code. On COMPLETED: optional summary. Example: ORDER_ALREADY_TERMINAL

Indexes: PK; (order_id, requested_at DESC). P7Y retention (compliance).

Sample data:

id	order_id	requested_by_actor	requested_by_role	reason	requested_at	status	status_detail
cocr_01HZ_01	ord_01HZ...A	support_agent_03	ORDER_SUPPORT	CUSTOMER_REQUESTED	2026-05-18 10:15:32	COMPLETED	NULL
cocr_01HZ_02	ord_01HZ...B	cust_01HX...B	CUSTOMER_SELF_SERVICE	CUSTOMER_REQUESTED	2026-05-18 14:00:33	COMPLETED	refund_requested
cocr_01HZ_03	ord_01HZ_TERMINAL	cust_01HX_T	CUSTOMER_SELF_SERVICE	CUSTOMER_REQUESTED	2026-05-18 09:00:00	REJECTED	ORDER_ALREADY_TERMINAL

Note: PAYMENT_TIMEOUT auto-cancellations do NOT write to this table (no API call). Timer-fire cancellations are visible only via customer_order.failure_reason and the OrderCancelled event.

8. Kafka events emitted

sophix.orders.cancelled

```
{
  "eventType": "OrderCancelled",
  "aggregateId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "timestamp": "2026-05-18T14:00:34.789Z",
  "payload": {
    "orderId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "customerId": "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
    "operatorCode": "WIK",
    "cancellationReason": "CUSTOMER_REQUESTED",
    "cancellationDetail": "Customer called support – changed mind, no longer interested",
    "cancellationActor": "support_agent_03",
    "cancelledFromPhase": "INSTALLING",
    "cascadeOutcome": {
      "subscriptionTerminated": true,
      "installWoCancelled": true,
      "refundRequested": true
    },
    "paymentReceivedAmount": 15000.00,
    "currency": "KES",
    "cancelledAt": "2026-05-18T14:00:34Z"
  }
}
```

sophix.orders.refund-requested

Canonical schema (also emitted by STEP-INSTALL and STEP-ACTIVATION on their respective failure paths). originatingStep discriminates.

```
{
  "eventType": "OrderRefundRequested",
  "aggregateId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "timestamp": "2026-05-18T14:00:33.412Z",
  "payload": {
    "orderId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "customerId": "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
    "operatorCode": "WIK",
    "subscriptionId": "sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q",
    "paymentReceivedAmount": 15000.00,
    "currency": "KES",
    "cancellationReason": "CUSTOMER_REQUESTED",
    "originatingStep": "STEP-CANCELLATION",
    "requestedAt": "2026-05-18T14:00:33Z"
  }
}
```

Consumers:

Consumer	Purpose
DD_NOT-01	On OrderCancelled: customer SMS+email with reason and (if refundRequested) refund timeline.
BIL-02-ADJ-01	On OrderRefundRequested: queue for Finance to process.
Finance	Refund processing dashboard.
CVM	Funnel cancellation tracking — split by cancellationReason and cancelledFromPhase to identify drop-off patterns.
NOC	cascadeOutcome flags with false values trigger alerts (cleanup failure during cancellation — ops investigation needed).

9. Step-pinned rules

ID	Rule
R-FUL-02-CAN-1	POST /api/orders/{id}/cancel correlates Camunda's OrderCancellationRequested message directly via POST /engine-rest/message — NOT through Kafka. Cancellation is a control-plane action that returns synchronous error codes to the caller (404, 409, 403, etc.); Kafka would lose this. Downstream notification events (OrderCancelled, OrderRefundRequested) ARE emitted on Kafka.
R-FUL-02-CAN-2	The cancel API rejects orders already ACTIVATED with 409 ORDER_ALREADY_ACTIVATED_USE_TERMINATE. Cancelling an activated subscription is a SUB-WF-TERMINATE-01 operation (different code path, different refund/proration rules, different audit chain). FUL-02 stops covering an order once current_phase = ACTIVATED.
R-FUL-02-CAN-3	order-cascade-cancellation cleans up in REVERSE creation order: install WO → subscription → refund. Each cleanup is conditional on the corresponding customer_order column being non-NULL. Skipping vs running is decided by NULL-check, not by phase — works regardless of which step interrupted the order.
R-FUL-02-CAN-4	Subscription termination via SUB-WF-TERMINATE-01 is async (its own saga). The cascade worker accepts the 202 and continues; doesn't wait for the saga to complete. The customer's OrderCancelled notification fires after the cascade returns (subscription terminate may still be running). Rationale: customer should see "cancelled" promptly; underlying cleanup completes async.
R-FUL-02-CAN-5	Refund request is emitted as a separate OrderRefundRequested event from OrderCancelled, even though both fire from the same worker. The split lets BIL-02-ADJ-01 subscribe only to refund events without filtering.
R-FUL-02-CAN-6	Auto-cancellations (PT7D payment timer fire) do NOT write to customer_order_cancellation_requests — no API call was made. They're visible in customer_order.failure_reason and the OrderCancelled event's cancellationActor = system_timer only. The table audits human-initiated requests.
R-FUL-02-CAN-7	When the BPMN's main subprocess is interrupted by OrderCancellationRequested, any pending message correlations (e.g., a late WorkOrderCancelled from the WO module canceling our install WO during the cascade) find no active subprocess. Camunda returns 400; the corresponding consumer DLQs the event. The cascade in progress is already handling the cleanup — no double-processing.
R-FUL-02-CAN-8	Cascade failure modes (e.g., WO module unreachable, SUB-WF-TERMINATE-01 5xx) do NOT block the order's transition to CANCELLED. The cascade worker logs the failure into cascadeOutcome flags and continues; the order is marked CANCELLED, the OrderCancelled event carries the partial-cleanup flags so NOC can alert. Manual cleanup happens out-of-band.
R-FUL-02-CAN-9	Phase-history rows during cancellation are appended by PhaseProjector like any other transition. The metadata field on the CANCELLED transition includes the full cancellationReason, cancellationActor, and cascadeOutcome for audit.